pfi

# Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

# SQL With sqlite3

WEEK 3 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# The desk lead's note: the book moves into a database

### The Desk Lead's Note

*The position file is moving into a database. Before any report comes out of it, give me the answers we already know, from SQL, each proved against pandas: the Energy bonds wider than 200 bp, the five largest positions, each sector's bonds, market value and average spread, the sectors holding at least 20 bonds, and the book's spread weighted by market value. Keep the queries where anyone can rerun them. Keep the database itself out of Git.*

The issuers and holdings are invented; every number describes the data. Nothing today goes to the network, so there is no key and no live cell.

```
Work folder pfi_course3_11_k4w7q2xr: 10 data files, Course 2's bondmath.py, marketdata.py as notebook 10 left it, 10 commits, tagged week1 and week2
.....................................   [100%]
42 passed in 0.50s
```

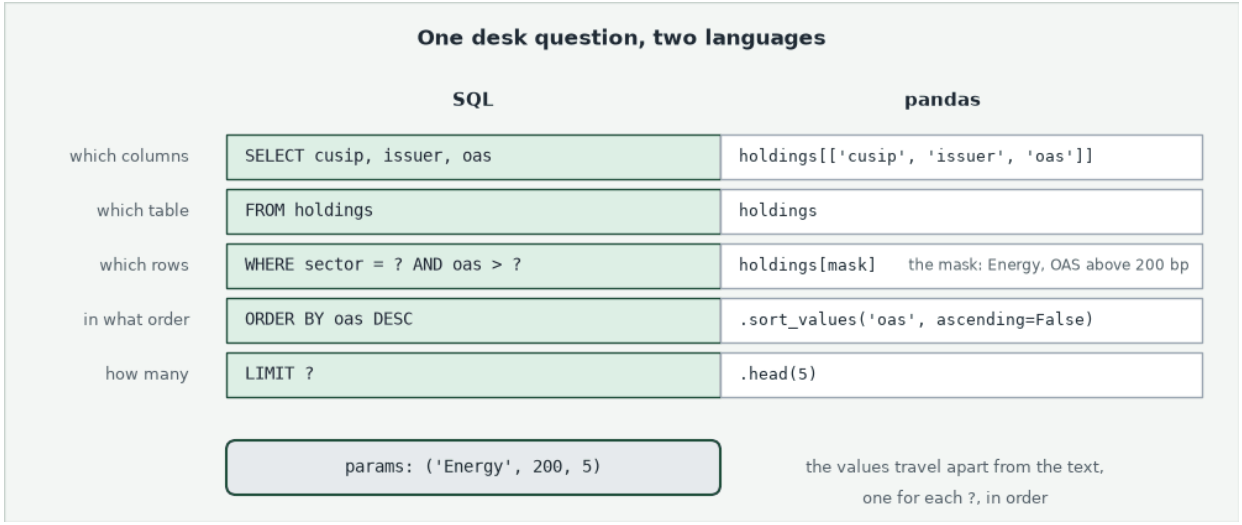
### Imports and the Holdings File

```
holdings = pd.read_csv('holdings.csv')
print(f"{len(holdings)} bonds as of {holdings.loc[0, 'as_of_date']}, {holdings.shape[1]} columns")
200 bonds as of 2026-09-30, 19 columns
```

**Every query sits beside the pandas you trust, and an assert proves they return the same rows.**

# 11.1 Why SQL, and why assistants write so much of it

Security masters, positions and trades often live in a database: named tables, one row per record, the shape of a CSV file. SQL is the language a program uses to ask a database for rows, and a request is a query.




A query names the result you want, clause by clause; the database works out how to get it. Assistants write SQL readily, and their SQL runs whether or not it is right. So every query today gets a pandas twin.

**SQL names the result you want, clause by clause; pandas is the answer key.**

# 11.2 sqlite3: a database in one file

The databases a desk reads usually run as a server you log in to. SQLite is a whole database in one file, with no server and no password, and Python ships with it.

### Which SQLite?

```
import sqlite3
print('SQLite', sqlite3.sqlite_version)
SQLite 3.50.4
```

### Two Lines Load a CSV

"sqlite3.connect(path)" opens the file, making it if it is missing, and returns a connection. "closing()" from "contextlib" closes the file at the end of the "with" block; "sqlite3's own with" does not.

```
from contextlib import closing
db_path = work / 'course3.db'
with closing(sqlite3.connect(db_path)) as connection:
    holdings.to_sql('holdings', connection, if_exists='replace', index=False)
    count = connection.execute('SELECT COUNT(*) FROM holdings').fetchone()[0]
print(count, 'rows in the table,', len(holdings), 'in the file')
200 rows in the table, 200 in the file
```

**Two lines put a CSV into a database file, and "COUNT(*)" reads its rows back: 200, the same as the file.**

# Is it text? The database you already have

Q. A CSV is text. What is the database file?

```
print((work / 'holdings.csv').read_bytes()[:16])
print(db_path.read_bytes()[:16])
b'as_of_date,cusip'
b'SQLite format 3\x00'
```

The CSV opens with its header row. The database opens with SQLite's own label: a binary file, read through "sqlite3", never in a text editor.

### The Database You Already Have

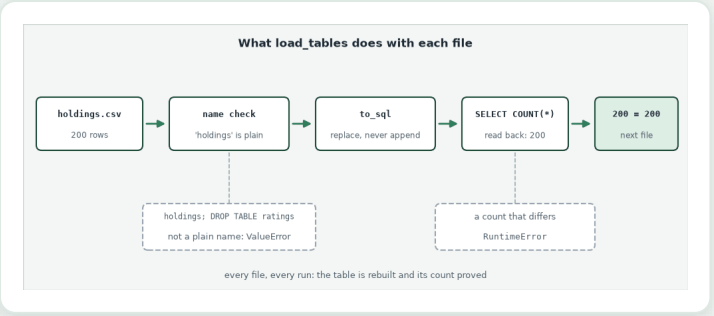
*SQLite's documentation calls it a database in "a single disk file" that can be copied from one machine to another, and says it is likely used more than all other database engines combined, in every smartphone among them. Every SQLite file begins with the 16 bytes you just printed.*

Story sources: SQLite, "Single-file Cross-platform Database", https://www.sqlite.org/onefile.html; "Most Widely Deployed and Used Database Engine", https://www.sqlite.org/mostdeployed.html; "Database File Format", https://www.sqlite.org/fileformat.html (all read 2026-10-04).

# 11.3 load_tables: five files, every count checked

Two lines worked for one file. The book has five, and a load should prove itself every time.

```
%%writefile -a marketdata.py
import sqlite3  # a database in one file, from day 11
from contextlib import closing  # closes a connection at the end of a with block, from day 11
def load_tables(db_path: Path, tables: dict[str, Path]) -> dict[str, int]:
    """Write each CSV file to a table of the SQLite database at db_path, and return each table's row count.
    Units: unchanged; dates stay ISO text (YYYY-MM-DD), which SQLite sorts and compares in date order.
    Convention: tables maps a table name to a CSV path; an existing table of that name is replaced. Each
    count is read back from the database with SELECT COUNT(*) and checked against the CSV's rows.
    Raises ValueError for a table name that is not a plain identifier (letters, digits, underscore), and
    RuntimeError if a table's count differs from its CSV's.
    """
    counts = {}
    with closing(sqlite3.connect(db_path)) as connection:
        for name, csv_path in tables.items():
            if not name.isidentifier():
                raise ValueError(f"table name {name!r} must be letters, digits, and underscores")
            frame = pd.read_csv(csv_path)
            frame.to_sql(name, connection, if_exists="replace", index=False)
            connection.commit()
            count = connection.execute(f"SELECT COUNT(*) FROM {name}").fetchone()[0]
            if count != len(frame):
                raise RuntimeError(f"table {name} has {count} rows but {Path(csv_path).name} has {len(frame)}")
            counts[name] = count
    return counts
```




"str.isidentifier()" is True for a plain name: letters, digits and underscores. "connection.commit()" saves the writes to the file.

# Five tables, and the test

## Five Tables

```
importlib.reload(md)
TABLES = {'holdings': 'holdings.csv', 'issuers': 'issuers.csv', 'ratings': 'ratings.csv',
          'benchmark': 'benchmark.csv', 'trades': 'trade_blotter.csv'}
table_counts = md.load_tables(db_path, TABLES)
table_counts
{'holdings': 200,
 'issuers': 150,
 'ratings': 18,
 'benchmark': 821,
 'trades': 374}
```

Each count was read back from the database and matched its file. "course3.db" lives in the work folder and is rebuilt on every run. The Treasury and spreads tables join tomorrow, in notebook 12. A table name goes into the SQL text itself, so only plain names pass; 11.5 shows what pasted text can do.

## The Test

In SQL, ";" ends one command and "DROP TABLE" deletes a table: the refused name carries a second command.

```
%%writefile -a test_marketdata.py
def test_load_tables_counts(tmp_path):
    counts = md.load_tables(tmp_path / "course3.db", {"holdings": HERE / "holdings.csv", "ratings": HERE / "ratings.csv"})
    assert counts == {"holdings": 200, "ratings": 18}
    assert md.load_tables(tmp_path / "course3.db", {"holdings": HERE / "holdings.csv"}) == {"holdings": 200}
    with pytest.raises(ValueError):
        md.load_tables(tmp_path / "course3.db", {"holdings; DROP TABLE ratings": HERE / "holdings.csv"})
```

```
.........................................   [100%]
43 passed in 0.50s
```

**"load_tables" writes each CSV to a table and refuses to finish unless every count matches its file.**

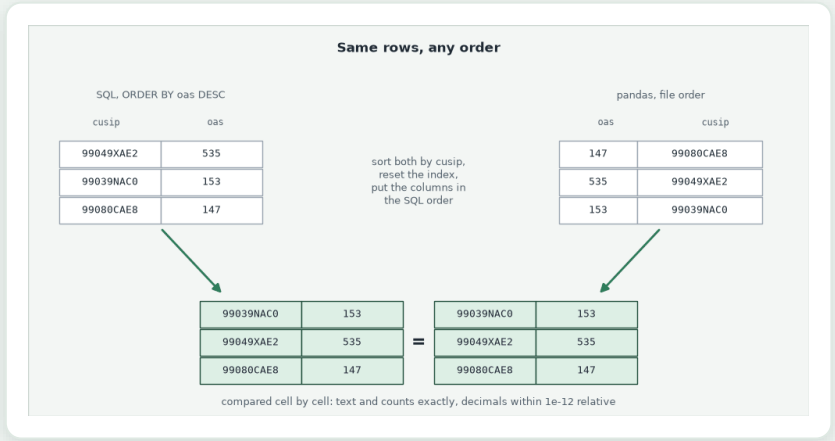
# 11.4 run_query, and the twin's proof

The tables are in; now ask them questions. "pd.read_sql_query(sql, connection, params=)" runs a query and returns its rows as a DataFrame.

```
%%writefile -a marketdata.py
def run_query(db_path: Path, sql: str, params: tuple = ()) -> pd.DataFrame:
    """Run one SQL query against the SQLite database at db_path and return the result.
    Units: as stored. Convention: values go in through ? parameters (`params`), never pasted into the SQL
    text with an f-string. The connection is closed when the query is done.
    Raises FileNotFoundError if db_path does not exist (sqlite3 would otherwise create an empty database).
    """
    if not Path(db_path).is_file():
        raise FileNotFoundError(f"no database file {Path(db_path).name}")
    with closing(sqlite3.connect(db_path)) as connection:
        return pd.read_sql_query(sql, connection, params=params)
```

"pd.testing.assert_frame_equal" is notebook 10's check of two tables; "check_dtype=False" lets a column of whole numbers equal one of decimals, and "rtol" and "atol" allow the last digits a sum in another order can move.

```
%%writefile -a marketdata.py
def assert_same_rows(left: pd.DataFrame, right: pd.DataFrame, sort_by: list[str]) -> None:
    """Check that two tables hold the same rows, in any order: a SQL result against its pandas equivalent.
    Units: unchanged. Convention: the same column names; both sorted by `sort_by`, index reset, columns in
    left's order; text and whole numbers equal exactly, floats within rtol=1e-12 and atol=1e-9 (SQLite and
    pandas may add in a different order). An integer column on one side may be a float column on the other.
    Raises AssertionError with pandas' message on the first difference.
    """
    if set(left.columns) != set(right.columns):
        raise AssertionError(f"columns differ: left only {sorted(set(left.columns) - set(right.columns))}, "
                             f"right only {sorted(set(right.columns) - set(left.columns))}")
    left_sorted = left.sort_values(sort_by).reset_index(drop=True)
    right_sorted = right[list(left.columns)].sort_values(sort_by).reset_index(drop=True)
    pd.testing.assert_frame_equal(left_sorted, right_sorted, check_dtype=False, rtol=1e-12, atol=1e-9)
```




A query returns its rows in its own order, and pandas keeps the file's. The twin's proof lines both up before it compares.

# SELECT, FROM and WHERE, each against pandas

Q. Four columns of every bond, from SQL and from pandas?

```
importlib.reload(md)
from_sql = md.run_query(db_path, 'SELECT cusip, issuer, sector, oas FROM holdings')
from_pandas = holdings[['cusip', 'issuer', 'sector', 'oas']]
md.assert_same_rows(from_sql, from_pandas, sort_by=['cusip'])
print('SQL and pandas: the same', len(from_sql), 'rows')
from_sql.head(3)
SQL and pandas: the same 200 rows
   cusip   issuer   sector  oas
0  99080CAE8   Bezane Finance   Financials  147
1  99049XAE2  Bezamyth Machinery   Industrials  535
2  99039NAC0   Bezoquil Freight  Transportation  153
```

| cusip | issuer | sector | oas |
|---|---|---|---|
| 99080CAE8 | Bezane Finance | Financials | 147 |
| 99049XAE2 | Bezamyth Machinery | Industrials | 535 |
| 99039NAC0 | Bezoquil Freight | Transportation | 153 |

Q. Which Energy bonds are wider than 200 bp? A "?" marks where a value goes, and the values travel in a tuple, in order; 11.5 shows why.

```
sql = 'SELECT cusip, issuer, oas FROM holdings WHERE sector = ? AND oas > ?'
from_sql = md.run_query(db_path, sql, ('Energy', 200))
mask = (holdings['sector'] == 'Energy') & (holdings['oas'] > 200)
md.assert_same_rows(from_sql, holdings.loc[mask, ['cusip', 'issuer', 'oas']], sort_by=['cusip'])
from_sql
```

| cusip | issuer | oas |
|---|---|---|
| 99145PAA0 | Praximer Energy | 468 |
| 99074WAB8 | Wyxundra Energy | 532 |
| 99074WAD4 | Wyxundra Energy | 332 |

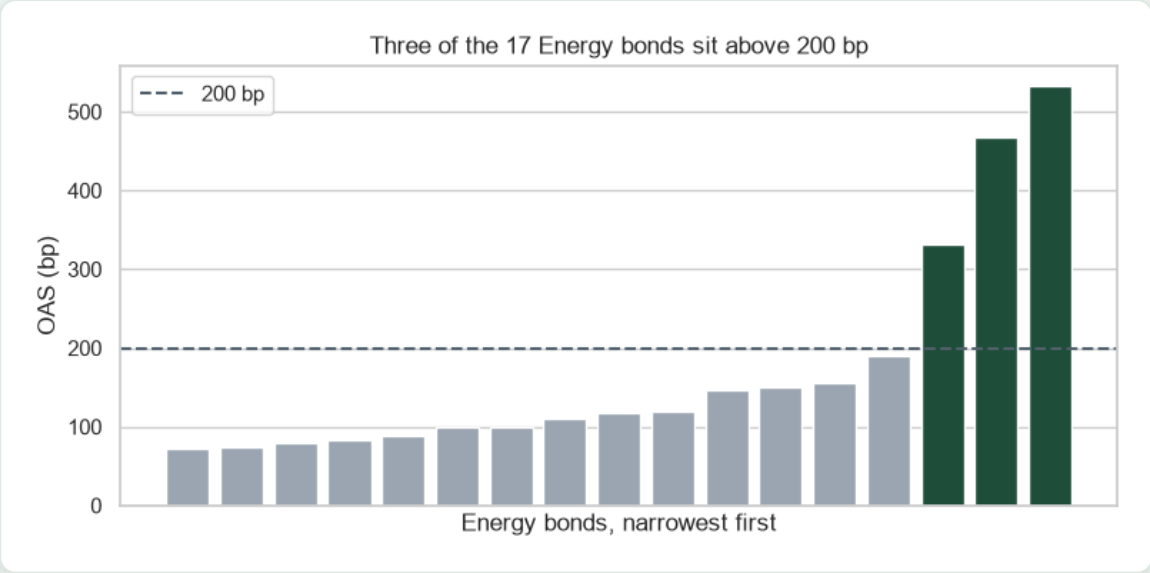
"SELECT *" returns every column. "OR" keeps a row that passes either test.

Excel side by side. The sheet: cusip in A, issuer in B, sector in C, rating in D, oas in E, market_value in F, the 200 bonds in rows 2 to 201.

| Job | SQL | pandas | Excel | Result |
|---|---|---|---|---|
| Energy wider than 200 bp | `WHERE sector = ? AND oas > ?` | the mask | `=FILTER(A2:A201,(C2:C201="Energy")*(E2:E201>200))` | the same 3 cusips |

"*" multiplies two TRUE or FALSE tests: 1 only when both are TRUE, so it works as AND.

# Three Energy bonds, then the five largest




**SQL, pandas and Excel return the same three bonds.**

Q. The five largest positions by market value?

```
sql = 'SELECT cusip, issuer, market_value FROM holdings ORDER BY market_value DESC LIMIT ?'
top_five = md.run_query(db_path, sql, (5,))
from_pandas = holdings.sort_values('market_value', ascending=False).head(5)
md.assert_same_rows(top_five, from_pandas[['cusip', 'issuer', 'market_value']], sort_by=['cusip'])
assert top_five['cusip'].tolist() == from_pandas['cusip'].tolist()
top_five.style.hide(axis='index').format({'market_value': '{:,.2f}'})
```

| cusip | issuer | market_value |
|---|---|---|
| 99080CAE8 | Bezane Finance | 5,435,500.00 |
| 99021VAC2 | Feskundra Systems | 5,422,700.00 |
| 99076YAC0 | Lorvebran Paper | 5,287,462.50 |
| 99129ZAB6 | Zarizal Industrial | 5,253,950.00 |
| 99144OAF3 | Lorvellix Energy | 5,241,650.00 |

"(5,)" is a tuple of one value: the comma makes it a tuple. Without "ORDER BY" a database promises no order at all.

# TAKE and SORTBY, and dates as text

## Excel Side by Side

| Job | SQL | pandas | Excel | Result |
|---|---|---|---|---|
| Five largest by market value | `ORDER BY market_value DESC LIMIT ?` | `.sort_values(ascending=False).head(5)` | `=TAKE(SORTBY(A2:A201,F2:F201,-1),5)` | the same 5 cusips, |
| | | | | in order |

## Dates Are Text

```
sql = 'SELECT cusip, issuer, maturity FROM holdings WHERE maturity < ? ORDER BY maturity'
early = md.run_query(db_path, sql, ('2030-01-01',))
print(len(early), 'bonds mature before 2030, the first on', early.loc[0, 'maturity'])
print(type(early.loc[0, 'maturity']))
34 bonds mature before 2030, the first on 2027-10-02
<class 'str'>
```

ISO text puts the year first, so text order is date order; notebook 12 shows when that breaks.

**Each clause has a pandas twin, and "assert_same_rows" proves every pair: the note's first two asks come straight from the database.**

# 11.5 A value pasted into the SQL text

Every value so far travelled in a tuple. Here is what happens when a value is pasted into the SQL text instead.

## A Ticker With a Quote

```
ticker = "O'BRN"
pasted = md.run_query(db_path, f"SELECT cusip FROM holdings WHERE ticker = '{ticker}'")
```

```
DatabaseError: Execution failed on sql 'SELECT cusip FROM holdings WHERE ticker =
'O'BRN'': near "BRN": syntax error
```

No bond has that ticker, yet the query never ran. The quote in "O'BRN" closed the SQL text early, and SQLite could not read the rest.

## The Same Question, as a Parameter

```
as_parameter = md.run_query(db_path, 'SELECT cusip FROM holdings WHERE ticker = ?', (ticker,))
print(len(as_parameter), 'bonds, and no error')
0 bonds, and no error
```

## The Test

"build_database" makes a small database for the tests that follow.

```
%%writefile -a test_marketdata.py
def build_database(folder: Path) -> Path:
    db_path = folder / "course3.db"
    md.load_tables(db_path, {"holdings": HERE / "holdings.csv", "ratings": HERE / "ratings.csv"})
    return db_path
def test_run_query_params(tmp_path):
    db_path = build_database(tmp_path)
    holdings = pd.read_csv(HERE / "holdings.csv")
    result = md.run_query(db_path, "SELECT COUNT(*) AS bonds FROM holdings WHERE sector = ?", ("Energy",))
    assert result.loc[0, "bonds"] == (holdings["sector"] == "Energy").sum()
    quoted = md.run_query(db_path, "SELECT * FROM holdings WHERE ticker = ?", ("O'BRN",))
    assert len(quoted) == 0
    with pytest.raises(FileNotFoundError):
        md.run_query(tmp_path / "missing.db", "SELECT 1")
```

```
.........................................   [100%]
44 passed in 0.50s
```

# 11.6 One row per sector

The note's next ask is a summary: one row per sector. "GROUP BY sector" makes one row per sector, and "COUNT(*)", "SUM()" and "AVG()" work over each sector's bonds. "AS" names a result column.

```
sql = '''SELECT sector, COUNT(*) AS bonds, SUM(market_value) AS market_value, AVG(oas) AS avg_oas
         FROM holdings GROUP BY sector ORDER BY market_value DESC'''
by_sector_sql = md.run_query(db_path, sql)
by_sector_sql.style.hide(axis='index').format(
    {'market_value': '{:,.2f}', 'avg_oas': '{:.2f}'}).bar(subset=['market_value'], color='#9AA5B1')
```

## The pandas Twin

```
by_sector_pandas = holdings.groupby('sector').agg(
    bonds=('cusip', 'count'), market_value=('market_value', 'sum'), avg_oas=('oas', 'mean'))
md.assert_same_rows(by_sector_sql, by_sector_pandas.reset_index(), sort_by=['sector'])
print(f"{by_sector_sql['bonds'].sum()} bonds, {by_sector_sql['market_value'].sum():,.2f} dollars")
200 bonds, 490,819,215.00 dollars
```

| sector | bonds | market_value | avg_oas |
|---|---|---|---|
| Materials | 26 | 71,701,617.50 | 182.42 |
| Financials | 26 | 68,219,672.50 | 172.31 |
| Consumer | 26 | 56,629,597.50 | 218.88 |
| Telecom | 21 | 55,848,270.00 | 182.38 |
| Industrials | 20 | 52,540,745.00 | 220.55 |
| Transportation | 20 | 48,822,897.50 | 126.50 |
| Health Care | 20 | 38,908,187.50 | 141.75 |
| Energy | 17 | 38,830,005.00 | 171.59 |
| Technology | 12 | 30,311,760.00 | 298.17 |
| Utilities | 12 | 29,006,462.50 | 177.92 |

# The pivot table, then HAVING

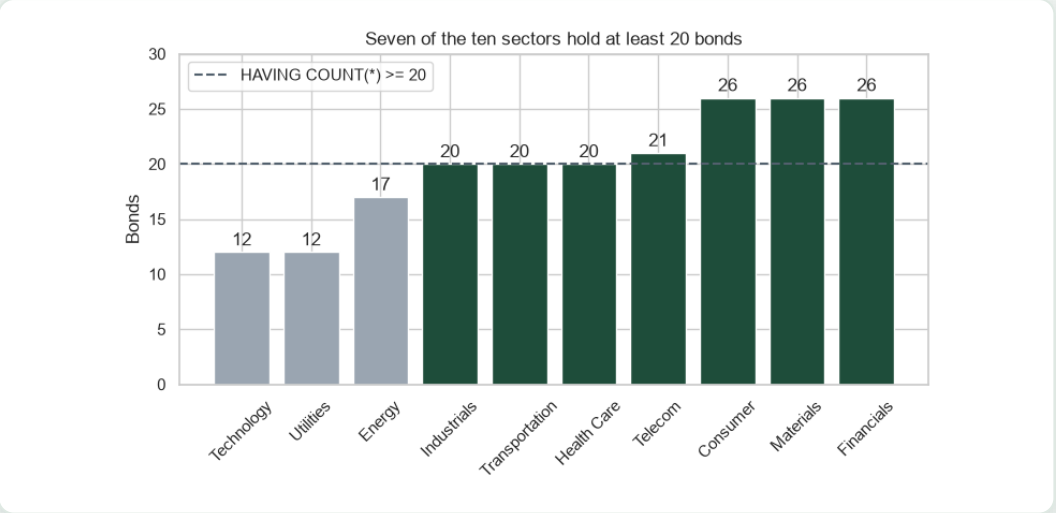
Excel side by side. A pivot table groups the same sheet. Insert, then PivotTable, with "sector" in Rows and "cusip" (Count), "market_value" (Sum) and "oas" (Average) in Values. The formulas need a summary block, the ten sectors in J2:J11.

| Job | SQL | pandas | Excel | Result |
|---|---|---|---|---|
| One row per sector | `GROUP BY sector` | `.groupby('sector').agg(...)` | the pivot table | 10 rows; |
| | | | | Grand Total 200 bonds, 490,819,215.0, 185.75 |
| Bonds per sector | `COUNT(*)` | `('cusip', 'count')` | `=COUNTIFS($C$2:$C$201,J2)` | Energy 17 |
| Market value | `SUM(market_value)` | `('market_value', 'sum')` | `=SUMIFS($F$2:$F$201,$C$2:$C$201,J2)` | Energy 38,830,005.0 |
| Average OAS | `AVG(oas)` | `('oas', 'mean')` | `=AVERAGEIFS($E$2:$E$201,$C$2:$C$201,J2)` | Energy 171.5882353 |

Q. Which sectors hold at least 20 bonds? "HAVING" keeps the sectors that pass a test on the group.

```
sql = '''SELECT sector, COUNT(*) AS bonds FROM holdings
         GROUP BY sector HAVING COUNT(*) >= ? ORDER BY sector'''
kept = md.run_query(db_path, sql, (20,))
counts = by_sector_pandas.reset_index()[['sector', 'bonds']]
md.assert_same_rows(kept, counts[counts['bonds'] >= 20], sort_by=['sector'])
print(len(kept), 'of', len(counts), 'sectors:', ', '.join(kept['sector']))
7 of 10 sectors: Consumer, Financials, Health Care, Industrials, Materials, Telecom, Transportation
```

Excel side by side. "=FILTER(J2:K11,K2:K11>=20)" on the summary block, its counts in K, returns the same 7 sectors.



# Two tests for the twin's proof

Excel side by side. "=COUNTIFS($C$2:$C$201,J2,$E$2:$E$201,">200")" counts with two tests: 3 for Energy, the three bonds "WHERE" found.

The first proves the order does not matter, the second that a difference is caught. "as_index=False" in the first keeps "sector" as a column, as ".reset_index()" does.

```
%%writefile -a test_marketdata.py
def test_assert_same_rows_ignores_order(tmp_path):
    db_path = build_database(tmp_path)
    sql_result = md.run_query(db_path, "SELECT sector, COUNT(*) AS bonds, SUM(market_value) AS market_value "
                                       "FROM holdings GROUP BY sector ORDER BY market_value DESC")
    holdings = pd.read_csv(HERE / "holdings.csv")
    pandas_result = (holdings.groupby("sector", as_index=False)
                     .agg(market_value=("market_value", "sum"), bonds=("cusip", "count")))
    md.assert_same_rows(sql_result, pandas_result, sort_by=["sector"])
def test_assert_same_rows_catches_difference():
    left = pd.DataFrame({"sector": ["Energy", "Utilities"], "bonds": [20, 15]})
    right = pd.DataFrame({"sector": ["Utilities", "Energy"], "bonds": [15, 21]})
    with pytest.raises(AssertionError):
        md.assert_same_rows(left, right, sort_by=["sector"])
    with pytest.raises(AssertionError):
        md.assert_same_rows(left, left.rename(columns={"bonds": "count"}), sort_by=["sector"])
```

```
.............................................   [100%]
46 passed in 0.50s
```

# 11.7 A market-value weighted average in SQL

"AVG(oas)" gives every bond the same say, 250,000 face and 5,000,000 face alike. The desk weights by market value, as Course 2's "bondmath.weighted_average" does: the sum of OAS times weight over the sum of the weights.

### By Sector, Against bondmath

```
sql = '''SELECT sector, SUM(oas * market_value) / SUM(market_value) AS weighted_oas,
                AVG(oas) AS avg_oas
         FROM holdings GROUP BY sector'''
weighted_sql = md.run_query(db_path, sql)
rows = []
for sector, bonds in holdings.groupby('sector'):
    rows.append({'sector': sector, 'avg_oas': bonds['oas'].mean(),
                 'weighted_oas': bondmath.weighted_average(bonds['oas'], bonds['market_value'])})
from_bondmath = pd.DataFrame(rows)
md.assert_same_rows(weighted_sql, from_bondmath, sort_by=['sector'])
```

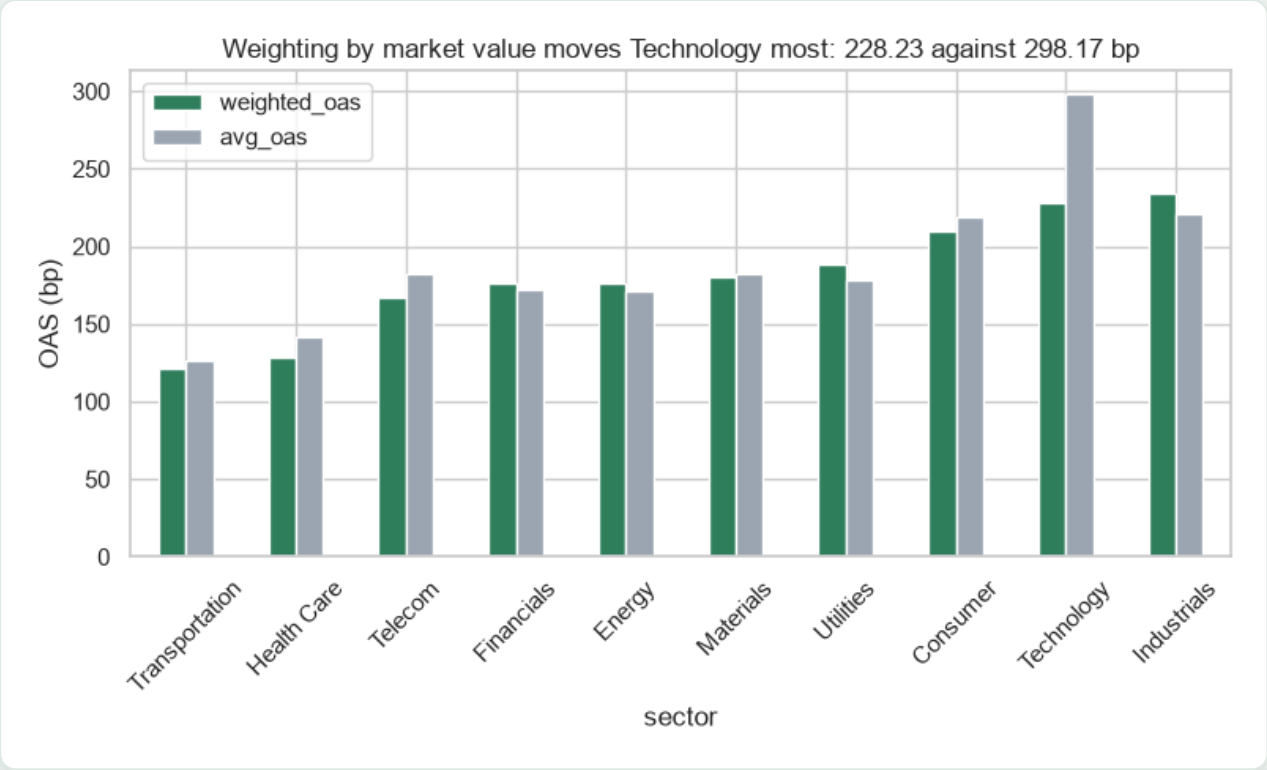
Excel side by side. In the summary block, sector in J2:

| Job | SQL | pandas | Result |
|---|---|---|---|
| Weighted OAS by sector | `SUM(oas * market_value) / SUM(market_value)` | `bondmath.weighted_average` | Energy 176.34941368150737, all ten equal |

```
=SUMPRODUCT(($C$2:$C$201=J2)*$E$2:$E$201*$F$2:$F$201)/SUMIFS($F$2:$F$201,$C$2:$C$201,J2)
```

The sector test gives TRUE or FALSE per row, and multiplied they count as 1 and 0, so only the sector's bonds add up.

# Weighted against simple, and the book in one row




## The Book in One Row

```
sql = '''SELECT SUM(oas * market_value) / SUM(market_value) AS weighted_oas,
                AVG(oas) AS avg_oas
         FROM holdings'''
book_sql = md.run_query(db_path, sql)
book_weighted = bondmath.weighted_average(holdings['oas'], holdings['market_value'])
print(f"SQL {book_sql.loc[0, 'weighted_oas']} bp, bondmath {book_weighted} bp")
print(f"simple average {book_sql.loc[0, 'avg_oas']} bp")
SQL 180.4609519769922 bp, bondmath 180.4609519769922 bp
simple average 185.75 bp
```

180.46 bp is the spread on Course 2's risk card, in Portfolio Aggregation and Scenarios. Course 1's Combining and Loading, Part 2 printed the same pair: simple 185.75, weighted 180.46.

**The book's market-value weighted OAS is 180.46 bp in SQL and in "bondmath".**

# 11.8 Commit the queries, not the database

The note's last two asks: keep the queries where anyone can rerun them, and keep the database out of Git.

### Is the Database Ignored?

```
!git -C "{work}" check-ignore -v course3.db
.gitignore:5:*.db              course3.db
```

Notebook 01's "*.db" line: any database file stays out of the record. "--" starts an SQL comment, and ";" ends each query.

### The Day's Queries in One File

```
%%writefile queries.sql
-- queries.sql: the day 11 queries on the course's synthetic book. Values go in as ? parameters.
-- four columns of every bond
SELECT cusip, issuer, sector, oas FROM holdings;
-- WHERE: one sector, spreads above a level (parameters: 'Energy', 200)
SELECT cusip, issuer, oas FROM holdings WHERE sector = ? AND oas > ?;
-- ORDER BY and LIMIT: the largest positions by market value (parameter: 5)
SELECT cusip, issuer, market_value FROM holdings ORDER BY market_value DESC LIMIT ?;
-- dates are ISO text, so < compares in date order (parameter: '2030-01-01')
SELECT cusip, issuer, maturity FROM holdings WHERE maturity < ? ORDER BY maturity;
-- GROUP BY: bonds, market value in dollars, and simple average OAS in bp by sector
SELECT sector, COUNT(*) AS bonds, SUM(market_value) AS market_value, AVG(oas) AS avg_oas
FROM holdings GROUP BY sector ORDER BY market_value DESC;
-- HAVING: sectors with at least this many bonds (parameter: 20)
SELECT sector, COUNT(*) AS bonds FROM holdings GROUP BY sector HAVING COUNT(*) >= ? ORDER BY sector;
-- market-value weighted OAS in bp by sector
SELECT sector, SUM(oas * market_value) / SUM(market_value) AS weighted_oas, AVG(oas) AS avg_oas
FROM holdings GROUP BY sector;
```

# What Git sees, and three files by name

### What Git Sees

```
!git -C "{work}" status --short --ignored
!git -C "{work}" diff --stat
 M marketdata.py
 M test_marketdata.py
?? queries.sql
!! __pycache__/
!! benchmark.csv
!! bondmath.py
!! course3.db
!! course3_excel_reference.csv
!! course3_fred_format_dgs10.json
!! course3_fred_format_dgs2.json
!! course3_synthetic_spreads.csv
!! holdings.csv
!! issuers.csv
!! ratings.csv
!! trade_blotter.csv
!! treasury_par_yields.csv
 marketdata.py   | 62 ++++++
```

# Bring it home, and what you can now do

Copy the work folder's "marketdata.py", "test_marketdata.py" and "queries.sql" into your own "my_bondmath" folder, with the two files the new tests read, test them there, and commit. The copy replaces your files of those names, so anything of your own must be in the work folder first. In a PowerShell terminal in VS Code, with the course's ".venv" active:

```
$work = "$env:TEMP\pfi_course3_11_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py", "$work\queries.sql" .
Copy-Item "$work\holdings.csv", "$work\ratings.csv" .
python -m pytest -q test_marketdata.py
git status --short
git add marketdata.py test_marketdata.py queries.sql
git commit -m "Notebook 11: load_tables, run_query, assert_same_rows, and queries.sql"
```

pytest shows 46 passed. "git status" lists the two data files as new: course data is never committed, so stage the three files by name. In Colab, download the three files from the work folder under "/tmp" instead. The work folder stays in your temp folder; the system clears it in time.

## You Can Now

Put CSV files into a SQLite database file with "load_tables", every row count checked.

Ask with "SELECT", "WHERE", "ORDER BY" and "LIMIT", and prove each answer against pandas with "assert_same_rows".

Summarise with "GROUP BY", "COUNT", "SUM" and "AVG", and filter groups with "HAVING".

Commit the queries and keep the database out of Git.

# Predict first: three queries

## Predict 1

Another value, pasted the same way: "NONE' OR '1'='1". What comes back?

A) no rows: no bond has that ticker

B) a syntax error, as "O'BRN" gave

C) every bond in the book

```
value = "NONE' OR '1'='1"
pasted = md.run_query(db_path, f"SELECT cusip FROM holdings WHERE ticker = '{value}'")
as_parameter = md.run_query(db_path, 'SELECT cusip FROM holdings WHERE ticker = ?', (value,))
print(f'pasted into the SQL: {len(pasted)} rows; as a parameter: {len(as_parameter)} rows')
```

## Predict 2

Move the count test into "WHERE": "WHERE COUNT(*) >= ?". What comes back?

A) the same 7 sectors

B) an error

C) all 10 sectors

```
sql = 'SELECT sector, COUNT(*) AS bonds FROM holdings WHERE COUNT(*) >= ? GROUP BY sector'
md.run_query(db_path, sql, (20,))
```

## Predict 3

Weight the book by face held instead: "SUM(oas * par_held) / SUM(par_held)". What does the query return, in bp? Write down a number.

```
sql = 'SELECT SUM(oas * par_held) / SUM(par_held) AS weighted_oas FROM holdings'
by_face = md.run_query(db_path, sql)
```

# The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.

1

## Docstring

Units, conventions, and what it must never do: what you asked for.

2

## Code back

Written for the course in the style of an AI answer. Read it.

3

## Test first

Assert what the docstring promises. Run it: it fails.

4

## Fix

Find the rule it broke, fix it, run the test again.

**Today: "SELECT grade, COUNT(*) AS bonds, AVG(oas) AS oas_bp" is a simple average, so High Yield: 339.3750 bp from the query, 334.7313 from bondmath.**

*Code that runs is not the same as code that is right.*

# The note, answered

| The note asked | Answer |
|---|---|
| Energy wider than 200 bp | 3 bonds, the same in SQL, pandas and Excel |
| The five largest positions | Bezane Finance first, 5,435,500.00 dollars |
| Each sector's bonds, market value and average spread | 10 sectors, 200 bonds, 490,819,215.00 dollars, tied to the pivot table |
| At least 20 bonds | 7 sectors |
| Spread weighted by market value | 180.46 bp, equal to bondmath |
| The queries | queries.sql committed, course3.db ignored |

Be proud: you learned a new language today and never once took it on trust.

**Next: SQL Joins and Time Series (notebook 12), where the tables meet and every join is counted.**